

Android Credential Leakage Detector

A modular Android APK security analysis framework designed to identify credential exposure, insecure storage patterns, weak network configurations, and runtime security risks using static and dynamic analysis techniques.

Overview

The framework analyzes Android APK files using a combination of:

- Static APK inspection
- Runtime evidence analysis
- Correlation-based risk evaluation

It is designed as a developer-oriented command-line security assessment tool for academic, research, and learning purposes.

Key Features

Static Analysis

- APK decompilation using Apktool
 - String extraction from:
 - strings.xml
 - Smali bytecode
 - Hardcoded credential detection
 - Sensitive token identification
 - Insecure storage pattern detection
 - Weak network configuration checks
 - Structured JSON finding generation
-

Dynamic Analysis

- Runtime log analysis
 - Runtime HTTP evidence parsing
 - Runtime credential leakage detection
 - Safe execution when runtime evidence is unavailable
-

Correlation Engine

- Correlates static and runtime findings
 - Escalates application risk based on combined evidence
 - Generates overall application risk assessment
-

Reporting

- JSON security reports
 - Terminal summary tables
 - Exportable PDF security assessment reports
-

Project Architecture

APK



Static Analysis

└─ APK Decompilation

└─ String Extraction

└─ Credential Detection

└─ Network Configuration Checks



Runtime Analysis

└─ Logcat Parsing

└─ Runtime HTTP Analysis

↓

Correlation Engine

↓

Risk Scoring

↓

JSON + PDF Reporting

Project Structure

android-credential-leakage-detector/

|

|— correlation/

| └─ correlation.py

|

|— dynamic_analysis/

| └─ parse_logcat.py

| └─ parse_pcap.py

|

|— static_analysis/

| └─ analyze_apk.py

| └─ detect_credentials.py

| └─ extract_strings.py

| └─ vuln_definitions.py

|

|— runtime_data/

| └─ logcat_runtime.txt

| └─ runtime_http.txt

```
|
|└─ reports/
|  |└─ generate_pdf.py
|  |└─ final_risk_report.json
|  |└─ static_findings.json
|  └─ final_security_report.pdf
|
|└─ samples/
|   └─ vulnerable.apk
|
|└─ main.py
|└─ requirements.txt
└─ README.md
```

Technologies Used

- Python 3
 - Apktool
 - ReportLab
 - Tabulate
 - JSON
 - Regex-based heuristic analysis
-

Installation

Clone Repository

```
git clone https://github.com/kvr585/android-credential-leakage-detector.git
```

```
cd android-credential-leakage-detector
```

Create Virtual Environment

```
python3 -m venv venv
```

```
source venv/bin/activate
```

Install Dependencies

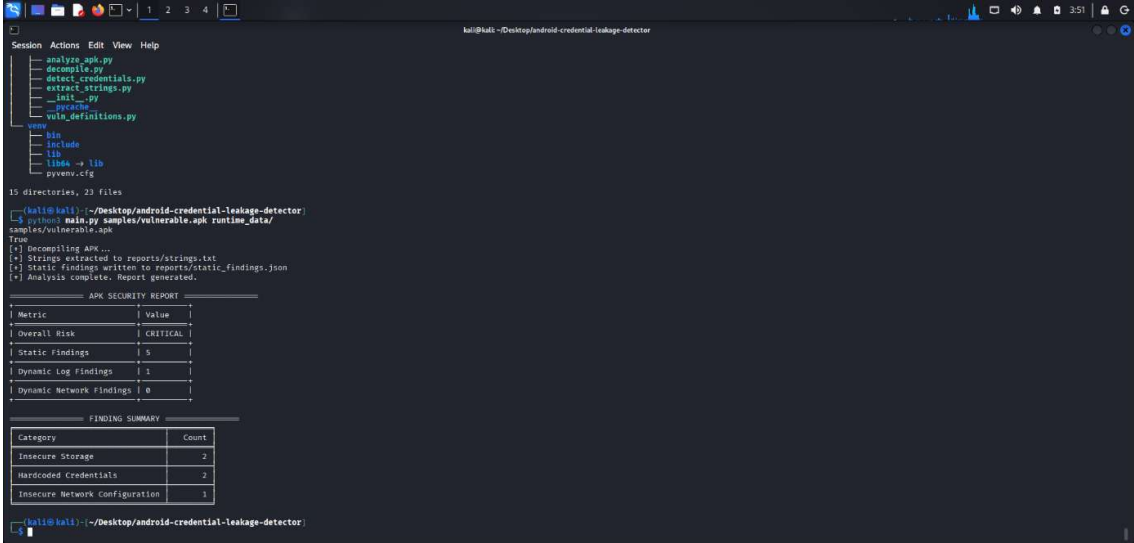
```
pip install -r requirements.txt
```

Usage

Run APK Analysis

```
python3 main.py samples/vulnerable.apk runtime_data/
```

Analysis Execution



```
Session Actions Edit View Help
├── analyze_apk.py
├── decompile.py
├── detect_credentials.py
├── extract_strings.py
├── _init_.py
├── _pycache_
├── vuln_definitions.py
└── venv
    ├── bin
    ├── include
    ├── lib
    ├── lib64 -> lib
    └── pyvenv.cfg

15 directories, 23 files

(hali@kali): ~/Desktop/android-credential-leakage-detector
$ python3 main.py samples/vulnerable.apk runtime_data/
samples/vulnerable.apk
True
[*] Decompiling APK ...
[*] Strings extracted to reports/strings.txt
[*] Static Findings written to reports/static_findings.json
[*] Analysis complete. Report generated.

===== APK SECURITY REPORT =====
| Metric | Value |
|-----|-----|
| Overall Risk | CRITICAL |
| Static Findings | 5 |
| Dynamic Log Findings | 1 |
| Dynamic Network Findings | 0 |
=====

===== FINDING SUMMARY =====
| Category | Count |
|-----|-----|
| Insecure Storage | 2 |
| Hardcoded Credentials | 2 |
| Insecure Network Configuration | 1 |
=====

(hali@kali): ~/Desktop/android-credential-leakage-detector
$
```

Generate PDF Security Report

```
python3 reports/generate_pdf.py
```

Example Findings

The framework can identify:

- Hardcoded credentials

- Sensitive storage patterns
- Plain HTTP usage
- Runtime credential exposure
- Insecure application behavior indicators

Static Analysis Findings



```

kali@kali:~/Desktop/android-credential-leakage-detector
Dynamic Network Findings | 0

FINDING SUMMARY
+-----+-----+
| Category | Count |
+-----+-----+
| Insecure Storage | 2 |
| Hardcoded Credentials | 2 |
| Insecure Network Configuration | 1 |
+-----+-----+

kali@kali:~/Desktop/android-credential-leakage-detector
$ cat reports/static_findings.json
{
  "category": "Insecure Storage",
  "severity": "MEDIUM",
  "location": "strings.txt:line 149",
  "evidence": "const-string v2, \\\"CREATE TABLE IF NOT EXISTS sqluser(user VARCHAR, password VARCHAR, credit_card VARCHAR);\\\""
},
{
  "category": "Hardcoded Credentials",
  "severity": "HIGH",
  "location": "strings.txt:line 154",
  "evidence": "const-string v2, \\\"INSERT INTO sqluser VALUES (\\\"admin\\\", \\\"password123\\\", \\\"1234567812345678\\\" );\\\""
},
{
  "category": "Hardcoded Credentials",
  "severity": "HIGH",
  "location": "strings.txt:line 174",
  "evidence": "const-string v2, \\\"INSERT INTO sqluser VALUES (\\\"john\\\", \\\"password123\\\", \\\"5555666677778888\\\" );\\\""
},
{
  "category": "Insecure Storage",
  "severity": "MEDIUM",
  "location": "strings.txt:line 7824",
  "evidence": "const-string v2, \\\"CREATE TABLE IF NOT EXISTS myuser(user VARCHAR, password VARCHAR);\\\""
},
{
  "category": "Insecure Network Configuration",
  "severity": "MEDIUM",
  "location": "strings.txt:line 8678",
  "evidence": "const-string v0, \\\"Register yourself at http://payatu.com to get your PIN and then login with that PIN!\\\""
}
}

kali@kali:~/Desktop/android-credential-leakage-detector

```

Generated Reports

Static Findings

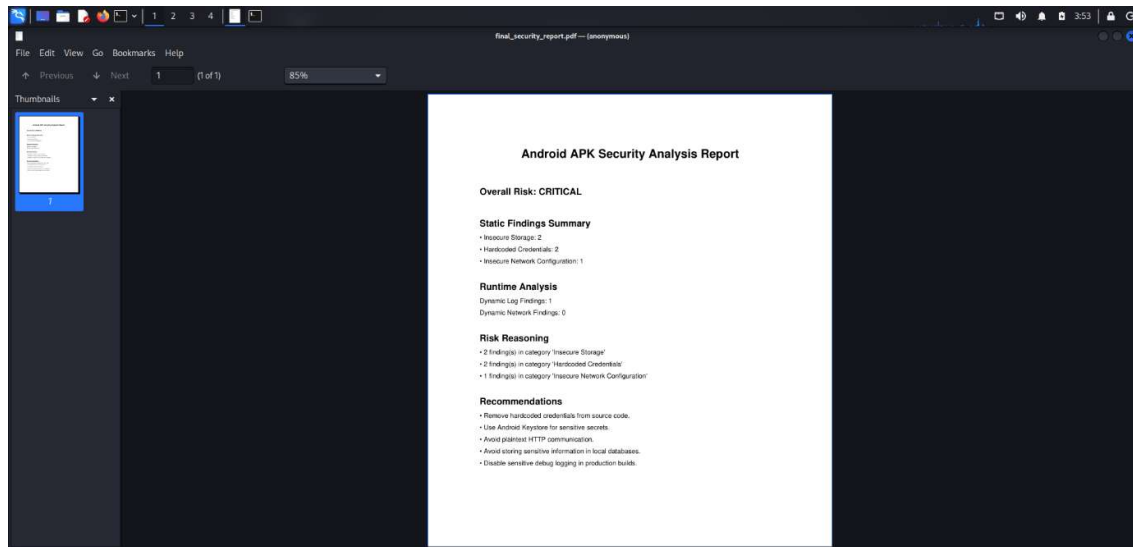
reports/static_findings.json

Final Risk Report

reports/final_risk_report.json

PDF Security Report

reports/final_security_report.pdf



Design Principles

- Static analysis is the primary inspection method
- Runtime analysis is evidence-based
- No artificial findings are generated
- Missing runtime artifacts are handled safely
- Findings are categorized and severity-tagged

Limitations

- Heuristic-based analysis
 - Regex-driven detection
 - Runtime analysis depends on external runtime artifacts
 - Encrypted traffic is not analyzed
 - No Android runtime instrumentation
-

Future Improvements

- Permission-risk correlation
 - Advanced runtime instrumentation
 - Confidence scoring system
 - HTML dashboard reporting
 - Enhanced heuristic detection
-

Intended Users

- Android security students
 - Cybersecurity researchers
 - Developers performing APK security checks
-

Disclaimer

This project is developed strictly for educational and research purposes and is not intended to replace commercial mobile security assessment platforms.

Author

Veera bhadhra

Sample Workflow

APK Input



Static APK Analysis



Runtime Evidence Parsing



Correlation Engine



Risk Evaluation



JSON + PDF Security Reports